main.py tests.py + 44kbjqe5z

```python
# ----------------------------------------------------------------
# File in src/main.py
# Main file wherein three functions were made for three test cases.
# Add, Subtract, and Multiply function were used to test.
# ----------------------------------------------------------------


def add(a, b):
    return a + b
"""Adds two numbers"""

def subtract(a, b):
    return a - b
"""Subtracts two numbers"""

def multiply(a, b):
    return a * b
"""Multiplies two numbers"""

# Workaround for using onecompiler to run tests.py
if __name__ == '__main__':
    import os
    os.system("python3 tests.py")
```

For one compiler, a workaround coding method is used. The main python file has three simple functions namely adding, subtracting, and multiplying two numbers.

main.py tests.py + 44kbjqe5z

```python
# ----------------------------------------------------------------
# File in tests/tests.py
# Tests the function of the main file.
# ----------------------------------------------------------------

import unittest
import main

def run_tests():
    print("--- Running Automated Tests ---")

    # TEST CASE 1: Addition
    result_add = main.add(10, 5)
    print(f"[TEST 1] Testing Addition: 10 + 5 = {result_add}")
    assert result_add == 15, "ERROR: Addition failed!"

    # TEST CASE 2: Subtraction
    result_sub = main.subtract(20, 8)
    print(f"[TEST 2] Testing Subtraction: 20 - 8 = {result_sub}")
    assert result_sub == 12, "ERROR: Subtraction failed!"

    # TEST CASE 3: Multiplication
    result_mult = main.multiply(7, 6)
    print(f"[TEST 3] Testing Multiplication: 7 * 6 = {result_mult}")
    assert result_mult == 42, "ERROR: Multiplication failed!"

    # When no error occurs, it prints out this string.
    print("\nSUCCESS: All 3 tests passed! OK")

# Run the tests.
if __name__ == '__main__':
    run_tests()
```

Testing file consists of three cases for each function and whenever the result of the function is not equal to the right output, it will display an error.

STDIN

Input for the program (Optional)

Output: 38 ms | 13.5 MB

```
--- Running Automated Tests ---
[TEST 1] Testing Addition: 10 + 5 = 15
[TEST 2] Testing Subtraction: 20 - 8 = 12
[TEST 3] Testing Multiplication: 7 * 6 = 42

SUCCESS: All 3 tests passed! OK
```

Simple enough, it shows that when testing that these sample runs were successful. Although, the given inputs were already defined.

tests.py M | main.py | README.md

lab_activities > reynoso_nathaniel_labactivity1 > tests > tests.py

```python
def run_tests():
    # TEST CASE 2: Subtraction
    result_sub = main.subtract(20, 8)
    print(f"[TEST 2] Testing Subtraction: 20 - 8 = {result_sub}")
    assert result_sub == 12, "ERROR: Subtraction failed!"

    # TEST CASE 3: Multiplication
    result_mult = main.multiply(7, 6)
    print(f"[TEST 3] Testing Multiplication: 7 * 6 = {result_mult}")
    assert result_mult == 42, "ERROR: Multiplication failed!"

    # When no error occurs, it prints out this string.
    print("\nSUCCESS: All 3 tests passed! OK")

# Run the tests.
if __name__ == '__main__':
    run_tests()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
* Executing task: /usr/bin/python3 /workspaces/CPE106L-4_E01_3T2526/lab_activities/reynoso_nathaniel_labactivity1/tests/tests.py

--- Running Automated Tests ---
[TEST 1] Testing Addition: 10 + 5 = 15
[TEST 2] Testing Subtraction: 20 - 8 = 12
[TEST 3] Testing Multiplication: 7 * 6 = 42

SUCCESS: All 3 tests passed! OK
* Terminal will be reused by tasks, press any key to close it.
```

bash tests

Python ...

I think I have successfully implemented it in GitHub Codespaces.